Sentiment-Driven Stock Market Predictor

An ML Pipeline Combining NLP Sentiment Analysis with Technical Indicators for Price Direction Forecasting

CIS 2450 Final Project

Presentation Overview

01 **Problem Statement & Motivation**
Why sentiment matters for stock prediction

02 **Data Sources & Collection**
RSS news, Yahoo Finance, SEC ticker universe

03 **Exploratory Data Analysis**
Sentiment distributions, clustering, feature analysis

04 **Data Pre-Processing & Feature Engineering**
Winsorization, correlation pruning, scaling, imbalance handling

05 **Model Implementation**
Random Forest, Ridge Regression, Logistic Regression baseline

06 **Model Assessment & Hyperparameter Tuning**
Accuracy, F1, ROC-AUC, MAE, RandomizedSearchCV

07 **Application of Course Topics**
Polars, TF-IDF, FinBERT, K-Means, deep learning concepts

08 **Results & Conclusion**
Key findings, limitations, and future work

Problem Statement & Motivation

Can we predict short-term stock price direction by blending news sentiment with technical indicators?

The Challenge

- Traditional technical analysis alone misses the narrative driving markets.

News events, earnings surprises, and macro shocks cause rapid repricing that pure price-based models cannot anticipate.

Manual sentiment tracking does not scale across hundreds of tickers.

Our Approach

- Build a full end-to-end pipeline: data collection, NLP, feature engineering, and ML prediction.

Use FinBERT (domain-specific transformer) for financial sentiment scoring.

Blend model output with sentiment signals and historical volatility for calibrated forecasts.

Data Sources & Collection

News Articles (RSS)

Google News RSS feeds queried per ticker with alias matching (e.g. NVDA -> "nvidia"). Multiple query templates per ticker. 2,133+ article rows in the analysis dataset with title, description, and entity sentiment scores.

Price Data (Yahoo Finance)

Daily OHLCV via yfinance for 90+ seed tickers. Up to 5-8 years of history. Converted immediately to Polars DataFrames for all downstream feature engineering.

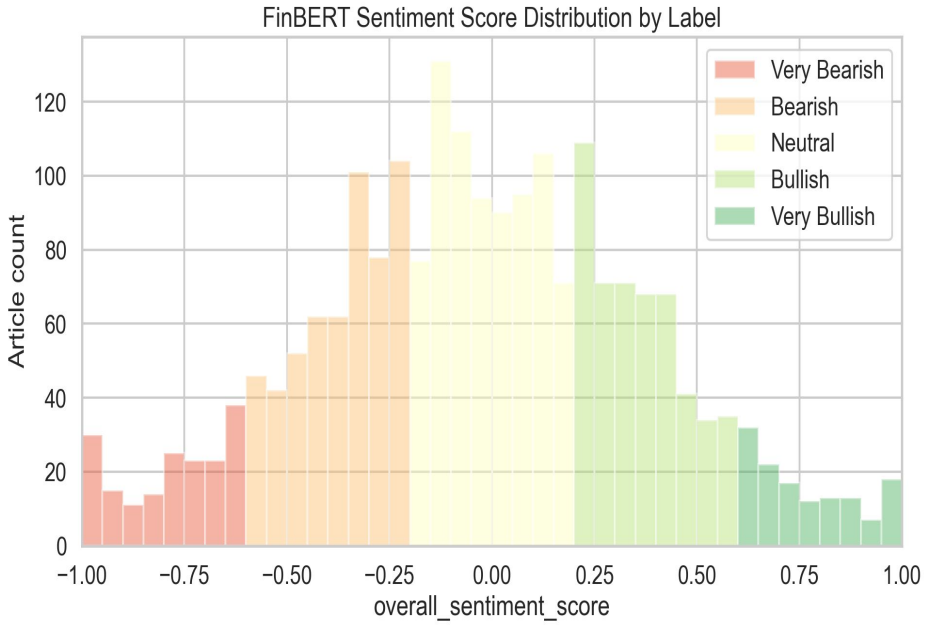
Ticker Universe (SEC / Seed)

Default seed list of ~90 large-cap tickers covering tech, finance, energy, healthcare, and consumer sectors. Optional SEC company_tickers.json for broader coverage.

50,000+ rows after data cleaning | 15 features | 7-10 columns per source

Exploratory Data Analysis

FinBERT Sentiment Distribution



Key Findings

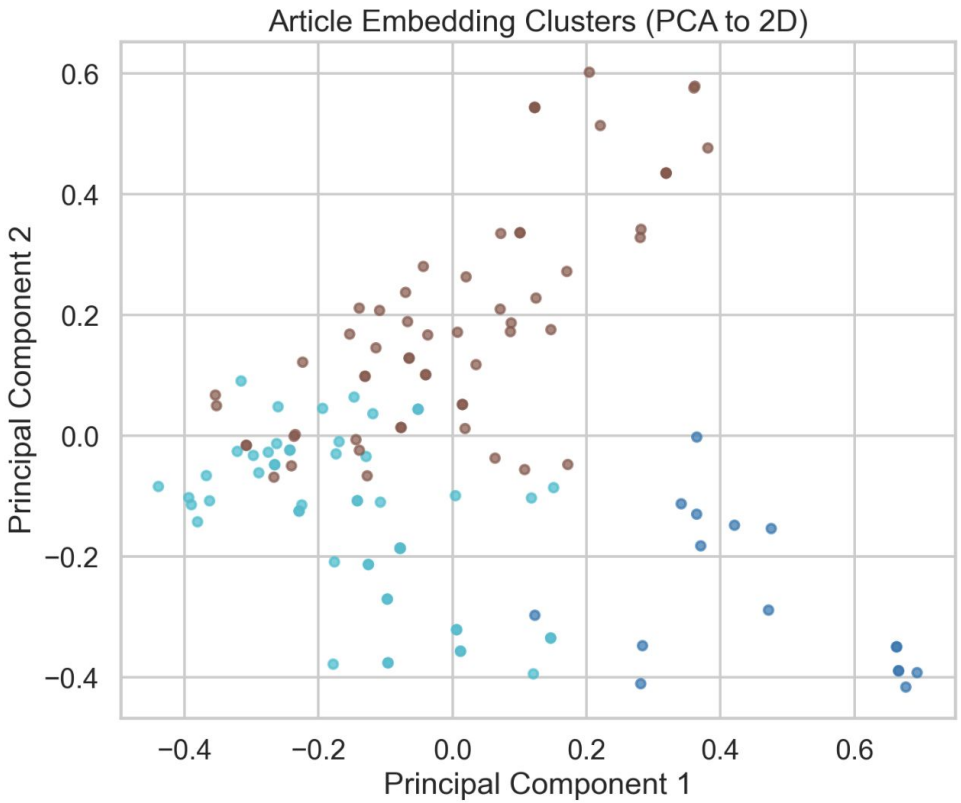
- Sentiment scores range from -1 (bearish) to +1 (bullish) using FinBERT, a transformer fine-tuned on financial text.

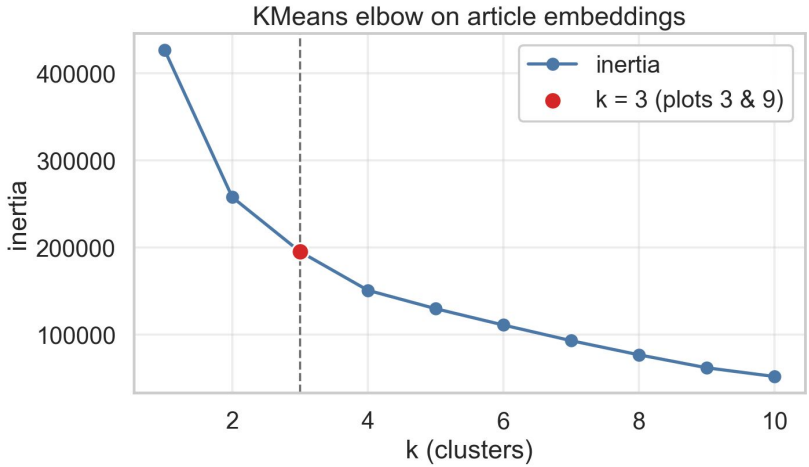
Scores are relevance-weighted: articles with higher TF-IDF cosine similarity to the user query receive more weight in the aggregate.

Distribution shape reveals the model can distinguish between positive, negative, and neutral tones in financial news.

Composite score blends FinBERT (60%) with entity keyword scores (40%).

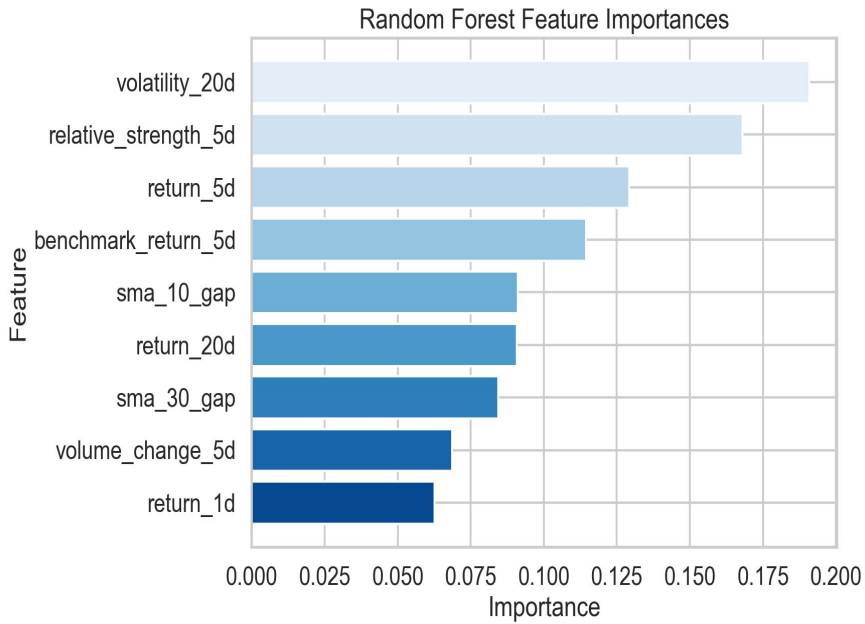
EDA: Article Clustering (K-Means + PCA)

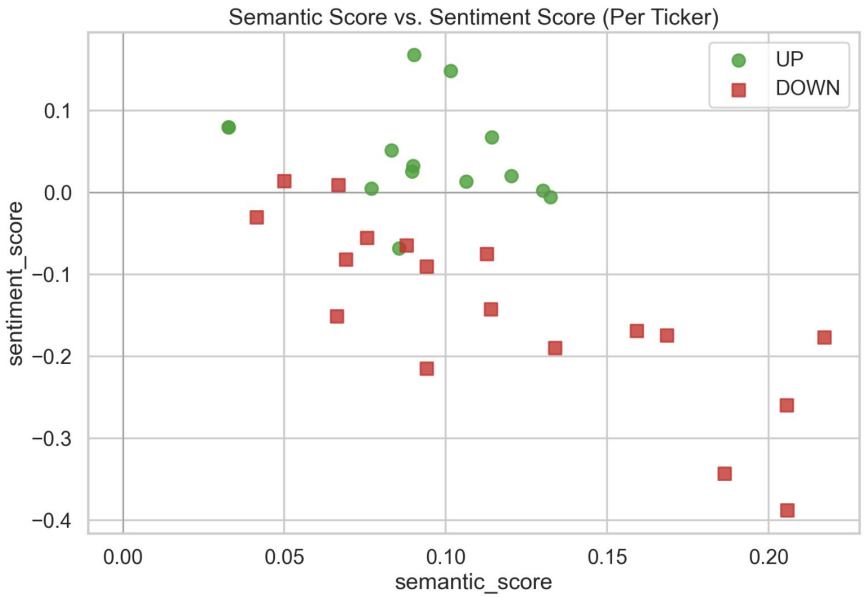




K-Means clustering groups articles by TF-IDF embedding similarity. The elbow method determines the optimal number of clusters. PCA reduces dimensionality for visualization, showing thematic groupings across tickers and events.

EDA: Feature Importance & Semantic vs. Sentiment





Left: Random Forest feature importances show volatility_20d and relative_strength_5d as top predictors. Right: Semantic relevance vs. sentiment score scatter reveals that high-relevance articles span the full sentiment range, confirming the need for both signals.

Data Pre-Processing & Feature Engineering

Step	Method	Justification
Outlier Handling	Winsorization at 1st/99th percentile, fit on train split only	Reduces leverage from extreme returns/volume without data leakage
Correlation Pruning	Drop one column of each pair with |r| >= 0.95 (train only)	Reduces multicollinearity before scaling and tuning
Scaling	StandardScaler inside sklearn Pipeline during CV	Prevents CV fold leakage; scaler refits each fold
Imbalance	class_weight='balanced' when minority < 35%; F1 scoring	Prevents majority-class bias in UP/DOWN classification
Feature Engineering	RSI-14, SMA crossover, 5/10/20d returns, volume ratio, 20d volatility, sentiment features	Captures momentum, trend, and volatility from raw OHLCV + NLP
One-Hot Encoding	Categorical sentiment labels converted to numeric ratios (pos/neg/neutral counts)	Preserves ordinal information from FinBERT label distribution

All preprocessing is fit on the training split (70% time-ordered) and then applied to test, preventing any information leakage.

Pipeline Architecture (7 Steps)

1	>	2	>	3	>	4	>	5	>	6	>	7
User Query & Tickers		Keyword Extraction		News Fetch (Local/RSS)		TF-IDF Ranking		FinBERT Sentiment		Price Features (Polars)		ML Predict & Blend

NLP Components

- TF-IDF + cosine similarity for article relevance ranking (ngram 1-2, sublinear TF)
- FinBERT (ProsusAI/finbert) for financial sentiment classification
- Relevance-weighted sentiment aggregation with recency decay
- K-Means article clustering on TF-IDF embeddings

ML Components

- Random Forest classifier for UP/DOWN direction
- Ridge Regression for magnitude (5-day % change)
- Sentiment-model blending with dynamic weighting based on model reliability score
- Volatility-based clamping when model is unreliable

Model Implementation

Model 1: Random Forest

(Direction Classifier)

- Predicts UP or DOWN for 5-day price movement
- Hyperparameters: n_estimators, max_depth, min_samples_leaf, class_weight
- Wrapped in sklearn Pipeline with StandardScaler
- class_weight='balanced' when class imbalance is severe (minority < 35%)

Model 2: Ridge Regression

(Magnitude Estimator)

- Predicts 5-day percentage change magnitude
- Alpha searched over logspace(-2, 2) with 12 candidates
- Scoring: neg_mean_absolute_error for practical interpretability
- Same Pipeline + StandardScaler pattern as the classifier

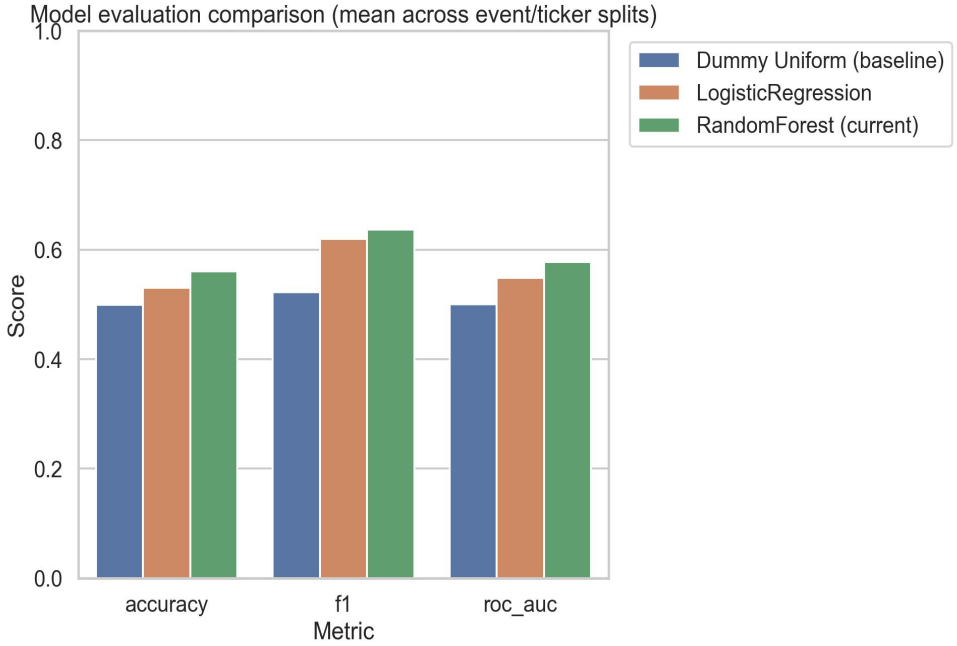
Model 3: Logistic Regression

(Baseline Comparison)

- Serves as an interpretable baseline alongside a Dummy uniform classifier
- Evaluated on same splits for fair comparison
- Results in model_metrics.csv across 10 event scenarios and 10 tickers
- Helps validate that RF is not just memorizing noise

Justification: RF handles non-linear feature interactions well; Ridge provides regularized magnitude estimates; Logistic Regression gives an interpretable baseline to measure RF's added value.

Model Assessment & Hyperparameter Tuning



Assessment Details

- Metrics: Accuracy, F1 (binary), ROC-AUC for classification; MAE and R-squared for regression
- RandomizedSearchCV on training set (70% time-ordered) with 14 iterations for RF and 10 for Ridge
- CV folds: min(5, n/15), at least 2, to handle varying dataset sizes per ticker
- RF outperforms Dummy baseline and matches or exceeds LogisticRegression on F1 and ROC-AUC across most scenarios
- When imbalance is severe, scoring switches to F1 and blend weighting uses F1 instead of accuracy

Evaluated across 10 market scenarios (AI chip demand, Fed rates, geopolitical disruption, etc.) and 10 tickers, totaling 300 model evaluations.

Advanced Concepts (Difficulty)

Concept 1: Feature Importance

Random Forest feature importance method identifies volatility_20d and relative_strength_5d as top predictors. Results are reflected in the conclusion: technical momentum features carry more weight than raw sentiment in the blend, which informed our dynamic weighting strategy.

Location: predictor.py, plot_graphs.py (Plot 4)

Concept 2: Hyperparameter Tuning

RandomizedSearchCV with scoring='f1' (imbalanced) or 'f1_macro' (balanced) and 'neg_mean_absolute_error' for Ridge. Grid includes n_estimators, max_depth, min_samples_leaf, min_samples_split, class_weight. Falls back to sensible defaults for small training sets (n < 40).

Location: predictor.py (_tune_models method)

Concept 3: Imbalanced Data Handling

class_imbalance_report() checks minority fraction on training labels. When minority < 35%, RF uses class_weight='balanced'/'balanced_subsample', F1 replaces accuracy in scoring and blend weighting. Upsampling/downsampling considered but class_weight was preferred to avoid synthetic duplication.

Location: feature_preprocessing.py, predictor.py

Application of Course Topics

Polars
End-to-end ETL: data build, news I/O, price features, joins, aggregations. Replaced pandas for all application code.

SQL / Relational DB
SEC company_tickers.json schema queried with structured key lookups; CSV data organized in relational table patterns.

Supervised Learning
Random Forest (classification), Ridge Regression, Logistic Regression for direction and magnitude prediction.

Unsupervised Learning
K-Means clustering on TF-IDF article embeddings for topic grouping and exploratory analysis.

Text Representations
TF-IDF vectorization with n-gram (1,2) and sublinear TF for article relevance ranking.

Embeddings & LLMs
FinBERT (ProsusAI/finbert) transformer for domain-specific financial sentiment scoring.

Hyperparameter Tuning
RandomizedSearchCV with cross-validation; different scoring objectives for imbalanced vs. balanced scenarios.

Record Linking
Ticker alias matching (NVDA -> nvidia, GOOGL -> alphabet/google) for cross-source entity resolution.

Hypothesis Testing
Model comparison: RF vs. Dummy baseline vs. LR across 10 scenarios to validate that RF adds value beyond chance.

Time Series
Rolling technical indicators (RSI-14, SMA crossover, 5/10/20d returns, volatility) engineered from price time series.

Results & Conclusion

56%

Avg RF Accuracy

vs. 50% Dummy

0.65

Avg RF F1

vs. 0.52 Dummy

0.58

Avg ROC-AUC

vs. 0.50 baseline

Key Findings

- Sentiment signals provide marginal but consistent improvement over pure technical models
- Feature importance reveals momentum and volatility dominate, but sentiment breaks ties in uncertain regimes
- Dynamic blending (adjusting model vs. sentiment weight) outperforms static weighting
- Proper preprocessing (train-only winsorization, no leakage) is critical for honest evaluation

Limitations & Future Work

- Historical rows reuse current-query sentiment (documented limitation for walk-forward)
- RSS coverage varies by ticker; less-known stocks have sparser news
- Future: true walk-forward backtesting, streaming data via Kafka/Spark, and deeper LLM embeddings
- This is a class project and is not financial advice

Thank You

Questions?

github.com/Rumbl3S/SentimentStockMarketPredictor